MACHINE LEARNING

Algorithmes de Clustering
A. Larhlimi

Classification Non Supervisée

Mars 2026 · Python · Scikit-learn · Iris · Wine · Digits

K-Means
DBSCAN
GMM
HAC
Spectral
UMAP
SOM

A. Larhlimi

Sommaire

01 Introduction au Clustering

02 Métriques d'évaluation

03 Algorithmes à centroïdes (K-Means, K-Medoids, K-Medians)

04 Algorithmes basés densité (DBSCAN, HDBSCAN, OPTICS)

05 Clustering Hiérarchique (HAC)

06 Modèles probabilistes (GMM)

07 Méthodes avancées (Spectral, Affinity Prop., SOM, UMAP)

08 Tableau comparatif global

09 Guide de sélection & Conclusion

A. Larhlimi

Introduction au Clustering

Qu'est-ce que le Clustering ?

Le clustering (classification non supervisée) est une branche fondamentale du machine learning. L'algorithme découvre lui-même la structure cachée dans les données — sans étiquettes fournies — en regroupant les observations similaires au sein de mêmes clusters et en séparant les observations dissimilaires.

Supervisé

Étiquettes fournies
Classes connues
Classification / Régression

Non Supervisé

Pas d'étiquettes
Structure auto-découverte
Clustering

Applications

- Segmentation de clients
- Détection d'anomalies
- Compression d'images
- Analyse de gènes
- Recommandation de contenu
- Traitement du langage naturel
- Analyse géospatiale
- Bioinformatique

A. Larhlimi

Score de Silhouette

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

a(i)

Distance moyenne de i à tous les autres points de son cluster (cohésion).

b(i)

Distance moyenne de i aux points du cluster voisin le plus proche (séparation).

s(i)

Résultat dans [-1, 1] : proche de 1 = bien classé ; ~0 = frontière ; négatif = mal classé.

Autres Métriques d'Évaluation

Inertie — WCSS (Within-Cluster Sum of Squares)

- Somme des distances au carré entre chaque point et son centroïde de cluster.
- Plus petite → meilleur regroupement intra-cluster.
- Utilisée principalement avec K-Means.
- Méthode Elbow : on trace l'inertie vs K ; le "coude" indique K optimal.

BIC — Bayesian Information Criterion

- Critère de sélection de modèle probabiliste pénalisant la complexité.
- Utile pour GMM : préférer le modèle minimisant le BIC.
- BIC = -2 × log-vraisemblance + k × log(n)

AIC — Akaike Information Criterion

- Similaire au BIC mais moins sévère sur la pénalité de complexité.
- AIC = -2 × log-vraisemblance + 2k
- Préférer AIC si l'objectif est la prédiction, BIC pour la sélection.

K-Means — Principe & Algorithme

L'algorithme de clustering le plus populaire. Partitionne n données en K clusters en minimisant l'inertie (variance intra-cluster). Repose sur le concept de centroïde : centre calculé comme la moyenne des points du cluster.

1

Initialisation

Sélectionner K centroïdes initiaux (K-Means++ recommandé).

2

Assignation

Affecter chaque point au centroïde le plus proche (distance euclidienne).

3

Mise à jour

Recalculer chaque centroïde = moyenne des points assignés.

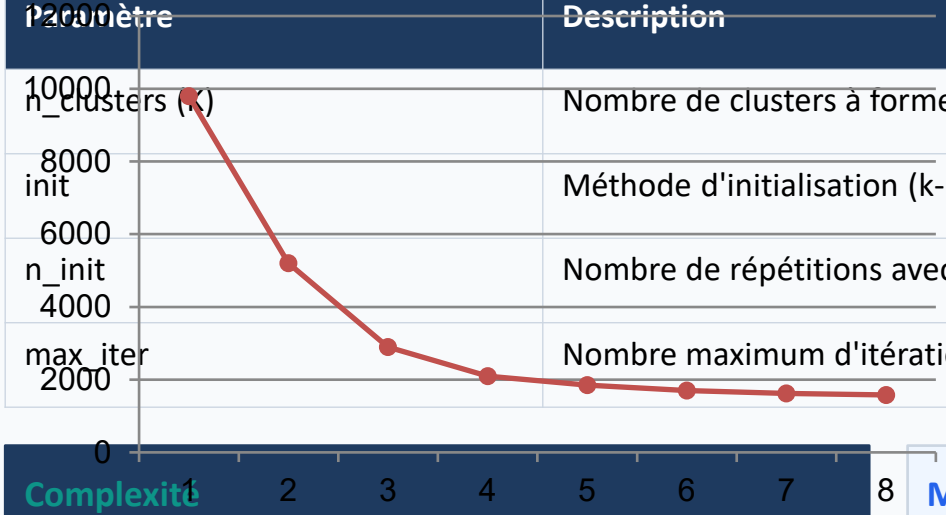
4

Convergence

Répéter étapes 2-3 jusqu'à ce que les assignations ne changent plus.

K-Means — Paramètres & Complexité

Paramètre	Description
n_clusters (K)	Nombre de clusters à former
init	Méthode d'initialisation (k-means++ recommandé)
n_init	Nombre de répétitions avec initialisations différentes
max_iter	Nombre maximum d'itérations



Complexité

Temporelle : O(n × K × d × i)
n = points, d = dimensions, i = itérations

Spatiale : O(n × K)

Méthode Elbow (choix de K)

On trace l'inertie en fonction de K. Le "coude" de la courbe indique le K optimal : au-delà de ce point, le gain marginal de qualité diminue fortement.

K-Means — Avantages & Limites

✓ Avantages

- Simple, rapide et scalable sur grands datasets
- Garantit la convergence
- Fonctionne bien sur clusters sphériques et bien séparés
- Faible empreinte mémoire
- Facile à implémenter et interpréter

✗ Limites

- K doit être fixé à l'avance
- Sensible aux outliers (centroïdes tirés vers eux)
- Suppose des clusters convexes et de taille similaire
- Peut converger vers un minimum local
- Non adapté aux clusters non-convexes

Cas d'usage : Segmentation client · Compression d'images · Quantification vectorielle · Pré-traitement

K-Medoids (PAM) — Partitioning Around Medoids

Variante de K-Means où les centres de clusters sont des points réels du dataset (médoïdes), non des moyennes fictives. Minimise la somme des dissimilarités entre chaque point et son médoïde.

	K-Means	K-Medoids
Centre	Moyenne (point fictif)	Point réel du dataset
Robustesse outliers	Faible	Élevée
Mesure de distance	Euclidienne uniquement	Toute métrique
Complexité	O(nKdi)	$O(K(n-K)^2)$

Avantages

- Robuste aux outliers et au bruit
- Fonctionne avec n'importe quelle métrique de distance
- Centres interprétables (points réels)

Limites

- Beaucoup plus lent que K-Means sur grands datasets
- K doit être fixé à l'avance
- Complexité quadratique

K-Medians — Robustesse par la Médiane

Similaire à K-Means mais utilise la médiane au lieu de la moyenne pour calculer les centres. Minimise la somme des distances L1 (Manhattan) — plus robuste aux valeurs extrêmes.

	K-Means (L2)	K-Medians (L1)
Fonction de coût	Somme des carrés	Somme valeurs absolues
Centre calculé	Moyenne	Médiane
Robustesse	Faible	Élevée
Distribution	Gaussienne	Non-gaussienne OK

Avantages

- Plus robuste aux outliers que K-Means
- Mieux adapté aux distributions non-gaussiennes
- La médiane ignore les valeurs extrêmes

Limites

- Convergence plus lente
- K doit être fixé à l'avance
- Moins de support dans les bibliothèques standard

ALGORITHMES BASÉS SUR LA DENSITÉ

DBSCAN · HDBSCAN · OPTICS

DBSCAN — Concepts Fondamentaux

DBSCAN regroupe les points densément connectés et marque les points isolés comme bruit. Détecte des clusters de forme arbitraire sans connaître K à l'avance.

ε (epsilon)	Rayon du voisinage d'un point.
MinPts	Nombre minimum de points dans le voisinage ε.
Point Central	A au moins MinPts voisins dans son rayon ε.
Point Frontière	Dans le rayon ε d'un point central, sans assez de voisins lui-même.
Point Bruit	Ni central ni frontière — outlier.

DBSCAN — Algorithme & Performances

1. Pour chaque point non visité, récupérer son voisinage ε.
2. Si ≥ MinPts voisins → point central → créer un nouveau cluster.
3. Étendre le cluster en ajoutant tous les points densément atteignables.
4. Sinon → marquer comme bruit (peut devenir frontière plus tard).

Complexité : $O(n \log n)$ avec K-d tree · $O(n^2)$ sans indexation spatiale

Avantages

- Détecte automatiquement le nombre de clusters
- Gère les clusters de forme arbitraire
- Robuste au bruit et aux outliers

Limites

- Sensible aux paramètres ε et MinPts
- Difficultés avec densités variables
- Mauvaises perf. en haute dimension

HDBSCAN — Extension Hiérarchique de DBSCAN

HDBSCAN construit une hiérarchie de clusters à différentes densités et extrait les clusters les plus stables. Beaucoup plus robuste au choix des paramètres que DBSCAN.

1.	**Distance de portée mutuelle**	Transformer les distances pour tenir compte de la densité locale.
2.	**Arbre couvrant minimum**	Construire un MST sur le graphe des distances mutuelles.
3.	**Hiérarchie de clusters**	Transformer le MST en dendrogramme de clusters.
4.	**Condensation**	Simplifier la hiérarchie en supprimant les petits clusters.
5.	**Extraction**	Sélectionner les clusters stables (maximisant la persistance).

Avantage clé : un seul paramètre principal (min_cluster_size) · Fournit des probabilités d'appartenance · Gère les densités variables.

OPTICS — Ordering Points To Identify Clustering Structure

OPTICS génère un ordonnancement des points révélant la structure de clustering à différents niveaux de densité. Produit un reachability plot encodant toute la structure hiérarchique.

Distance centrale

Distance au MinPts-ième voisin le plus proche.

Distance de portée

max(core-distance(o), dist(o, p))

Reachability Plot

Visualisation de la structure : les vallées correspondent aux clusters.

Avantages

- Fonctionne sans fixer ε
- Détecte clusters imbriqués et densités variables
- Analyse visuelle riche via reachability plot

Limites

- Ne produit pas directement des étiquettes
- Plus lent que DBSCAN
- Interprétation parfois subjective

Cas d'usage : Analyse exploratoire · Données avec hiérarchies de densité · Détection de sous-clusters

A. Larhlimi

CLUSTERING HIÉRARCHIQUE

HAC — Hierarchical Agglomerative Clustering

Clustering Hiérarchique Agglomératif (HAC)

Approche bottom-up : commence avec chaque point comme son propre cluster, puis fusionne progressivement les deux clusters les plus similaires jusqu'à obtenir un seul cluster racine. Résultat : un dendrogramme.

Méthode	Distance entre clusters	Comportement
Single	Distance minimale entre paires	Clusters allongés, sensible au bruit
Complete	Distance maximale entre paires	Clusters compacts, sensible outliers
Average	Moyenne de toutes les paires	Compromis Single/Complete
Ward	Minimise l'augmentation de variance	Clusters sphériques — le plus utilisé

Complexité

Temporelle : $O(n^3)$ / $O(n^2 \log n)$
Spatiale : $O(n^2)$

Dendrogramme

Coupe le dendrogramme à la hauteur souhaitée pour obtenir K clusters. Visualise toute la hiérarchie.

HAC — Avantages, Limites & Cas d'Usage

Avantages

- Pas besoin de spécifier K à l'avance
- Fournit une hiérarchie complète des clusters
- Déterministe (pas d'aléatoire)
- Résultat visualisable via le dendrogramme
- Supporte diverses métriques de distance

Limites

- Pas scalable pour de très grands datasets ($O(n^2)$ mémoire)
- Irréversible : mauvaise fusion initiale non corrigeable
- Sensible aux outliers (surtout single & complete linkage)
- Coût computationnel élevé
- Frontières de clusters parfois arbitraires

Cas d'usage : Phylogénétique · Analyse de documents · Segmentation d'images · Analyse de gènes · Bioinformatique

MODÈLES PROBABILISTES

Gaussian Mixture Models (GMM)

GMM — Gaussian Mixture Models

GMM suppose que les données sont générées par un mélange de K distributions gaussiennes. Effectue un clustering probabiliste (soft assignment) : chaque point appartient à chaque cluster avec une certaine probabilité.

$$p(x) = \Sigma_{k=1}^{K} \pi_k \times \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

π_k

Poids (probabilité a priori) du k-ième composant

μ_k

Vecteur moyenne du k-ième gaussien

Σ_k

Matrice de covariance du k-ième gaussien

Type covariance	Description	Flexibilité
full	Chaque composant a sa propre matrice	Maximale
tied	Tous partagent la même matrice	Moyenne
diag	Matrices diagonales	Réduite
spherical	Variance scalaire par composant	Minimale

GMM — Algorithme EM (Expectation-Maximization)

Étape E — Expectation

Calculer les responsabilités : probabilité a posteriori que chaque point appartienne à chaque composant gaussien.
$\gamma(z_{nk}) = \pi_k \times \mathcal{N}(x_n|\mu_k,\Sigma_k) / \Sigma_j \pi_j \times \mathcal{N}(x_n|\mu_j,\Sigma_j)$

Étape M — Maximization

Mettre à jour les paramètres (π_k, μ_k, Σ_k) pour maximiser la log-vraisemblance en utilisant les responsabilités calculées à l'étape E. Répéter jusqu'à convergence.

Avantages

- Clustering probabiliste (soft assignment)
- Modélise des clusters elliptiques
- Fournit des intervalles de confiance

Limites

- Suppose une distribution gaussienne
- Sensible à l'initialisation (minimum local)
- K doit être spécifié · Peut diverger

Cas d'usage : Parole · Vision par ordinateur · Segmentation d'images · Finance (modélisation des rendements)

A. Larhlimi

MÉTHODES AVANCÉES

Spectral · Affinity Propagation · SOM · UMAP+K-Means

Spectral Clustering — Théorie des Graphes

Exploite la théorie des graphes spectraux pour détecter des clusters de formes arbitraires. Transforme les données dans un espace où les clusters deviennent linéairement séparables via les vecteurs propres du Laplacien.

1. Construire le graphe de similarité (RBF ou k-NN).
2. Calculer la matrice W et le Laplacien : $L = D - W$
3. Calculer les k plus petits vecteurs propres de L.
4. Projeter les données dans l'espace des vecteurs propres.
5. Appliquer K-Means dans ce nouvel espace.

Avantages

- Clusters de formes arbitraires (y compris concentriques)
- Pas d'hypothèse sur la forme des clusters

Limites

- $O(n^3)$ — pas scalable · K doit être spécifié
- Sensible au paramètre d'affinité

A. Larhlimi

Affinity Propagation — Échange de Messages

Algorithme basé sur l'échange de messages entre toutes les paires de points. Découvre automatiquement le nombre de clusters en identifiant des exemplaires (points représentatifs).

Responsabilité r(i, k)

Mesure combien le point k est adapté comme exemplaire pour i (compétition avec d'autres candidats).

Disponibilité a(i, k)

Mesure combien il est approprié pour i de choisir k comme exemplaire (support des autres points).

Paramètre	Description
damping (λ)	Facteur d'amortissement ∈ [0.5, 1) — évite les oscillations
preference	Préférence initiale — contrôle le nombre de clusters

Avantages

- K déterminé automatiquement
- Centres = points réels · Déterministe

Limites

- $O(n^2)$ temps et mémoire · Pas scalable
- Sensible à la préférence

A. Larhlimi

Self-Organizing Maps (SOM) — Kohonen

Réseau de neurones non supervisé qui apprend à projeter des données de haute dimension sur une grille 2D (ou 1D) tout en préservant la topologie des données originales.

Architecture

- Couche d'entrée : vecteurs de données
- Couche de sortie : grille m×n de neurones
- Chaque neurone possède un vecteur de poids de même dimension que les données

Apprentissage

1. Trouver le BMU (Best Matching Unit)
2. Mettre à jour poids du BMU + voisins
3. Décroître α et rayon de voisinage

U-Matrix

Visualise les distances entre neurones voisins. Zones sombres = frontières entre clusters. Zones claires = régions homogènes.

Avantages

Préserve la topologie · Visualisation 2D intuitive · Haute dimension OK

Limites

Hyperparamètres nombreux · Frontières parfois floues · Convergence non garantie

A. Larhlimi

UMAP + K-Means — Clustering en Haute Dimension

UMAP (Uniform Manifold Approximation and Projection) est une technique de réduction de dimension non linéaire qui préserve la structure locale ET globale. Combiné avec K-Means pour clustériser efficacement des données de très haute dimension.

```
Données HD (n×d) → UMAP → Données 2D (n×2) → K-Means → Clusters
```

	UMAP	t-SNE
Vitesse	Rapide	Lent
Structure globale	Préservée	Peu préservée
Déterminisme	Semi-déterministe	Non déterministe
Clustering	Oui — applicable	Limité

Avantages

- Très efficace pour données de haute dimension
- Préserve structure locale ET globale · Rapide · Scalable

Cas d'usage

- scRNA-seq · NLP · Analyse génomique · Classification d'images

La

VUE D'ENSEMBLE

Tableau Comparatif & Guide de Sélection

Tableau Comparatif Global (1/2)

Algorithme	Type	K requis	Forme clusters	Scalabilité
K-Means	Centroïde	Oui	Convexe	Très haute
K-Medoids	Centroïde	Oui	Convexe	Moyenne
K-Medians	Centroïde	Oui	Convexe	Moyenne
DBSCAN	Densité	Non	Arbitraire	Haute
HDBSCAN	Densité	Non	Arbitraire	Haute
OPTICS	Densité	Non	Arbitraire	Moyenne

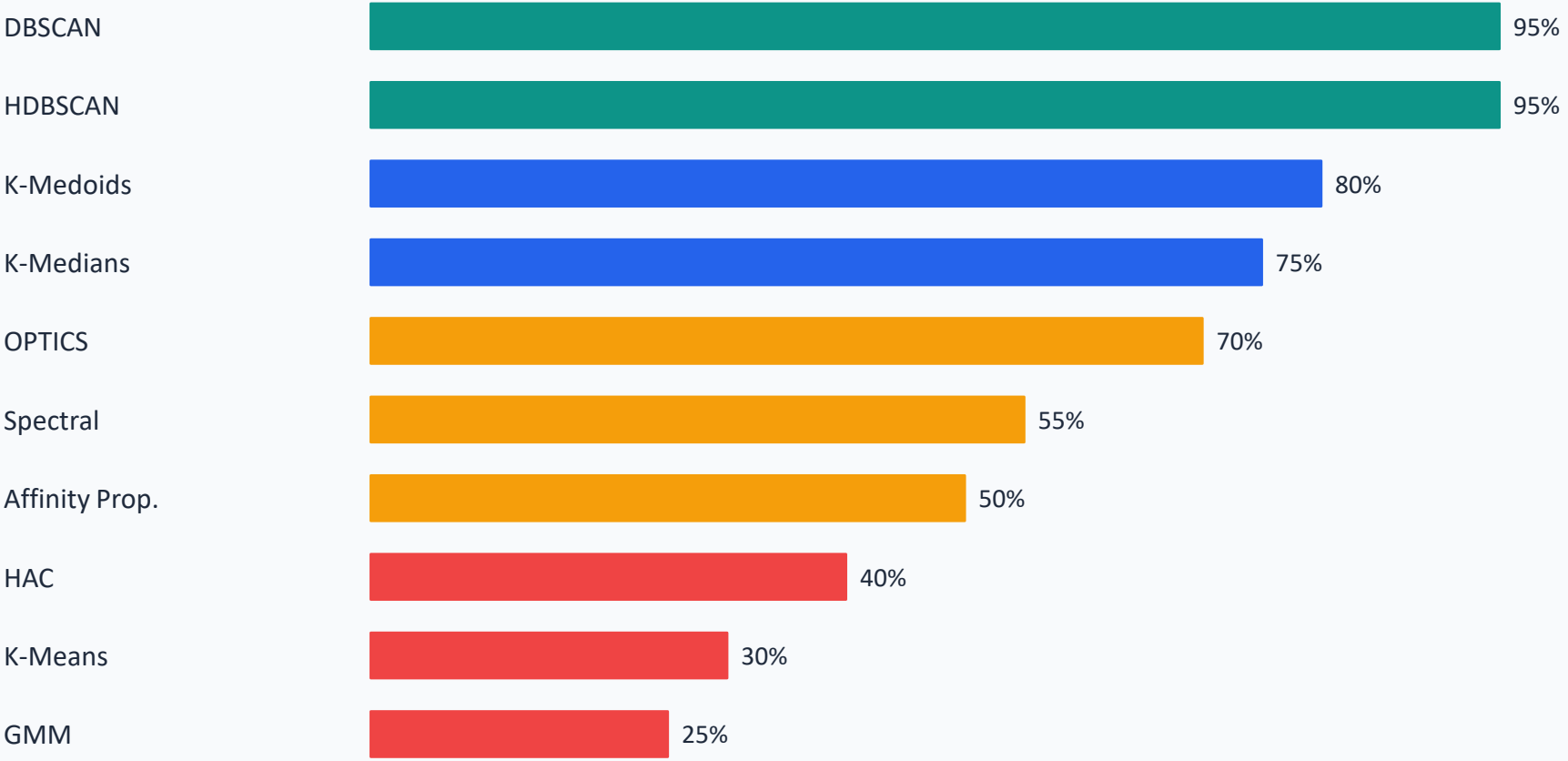
Tableau Comparatif Global (2/2)

Algorithme	Type	K requis	Outliers	Complexité
HAC	Hiérarchique	Non*	Sensible	$O(n^3)$
GMM	Probabiliste	Oui	Sensible	O(nKdi)
Spectral	Graphe	Oui	Modéré	$O(n^3)$
Affinity Prop.	Message	Non	Modéré	$O(n^2)$
SOM	Neuronal	Non*	Modéré	$O(n \times m^2 \times i)$
UMAP+K-Means	Hybride	Oui	Modéré	O(n log n)

** Non requis mais peut être déduit du dendrogramme / de la grille.*

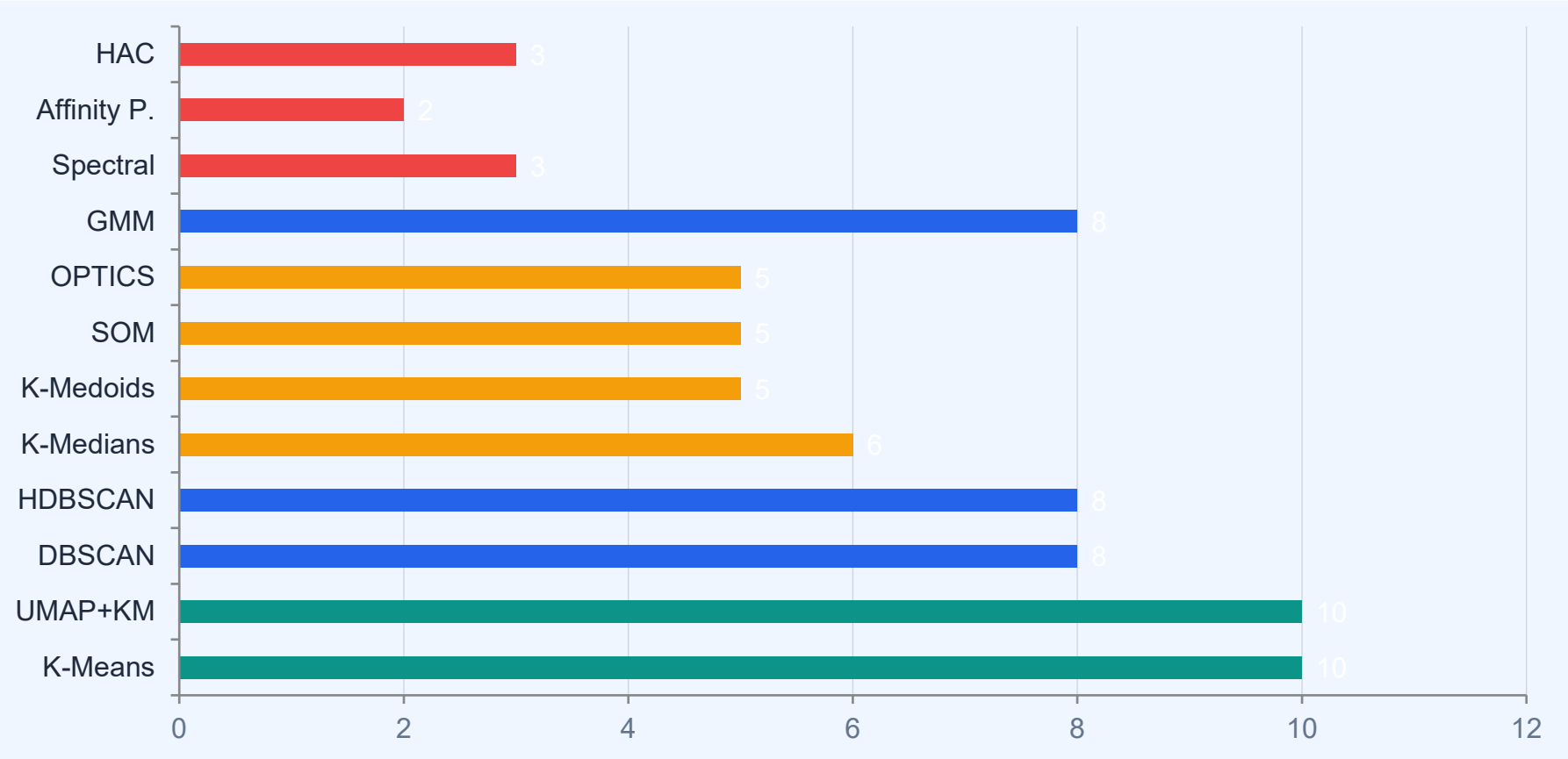
A. Larhlimi

Comparaison : Robustesse aux Outliers



A. Larhlimi

Comparaison : Scalabilité & Complexité



A. Larhlimi

Guide de Sélection de l'Algorithme

Haute dimension — Données de haute dimension (>50) ? — OUI → UMAP + K-Means ou SOM

NON ↓

K non connu — Nombre de clusters K connu ? — OUI → DBSCAN · HDBSCAN Affinity Prop. · HAC

NON ↓

Formes arbitraires — Clusters de formes arbitraires ? — OUI → DBSCAN · HDBSCAN Spectral

NON ↓

Bruit/outliers — Beaucoup de bruit / outliers ? — OUI → DBSCAN · K-Medoids K-Medians

A. Larhlimi

Guide de Sélection — Conditions Avancées

Besoin de probabilités d'appartenance ?	**OUI : GMM ou HDBSCAN**
	NON : Suite...
Besoin d'une hiérarchie de clusters ?	**OUI : HAC · HDBSCAN · OPTICS**
	NON : Suite...
Données très volumineuses (n > 100K) ?	**OUI : K-Means · UMAP+K-Means · HDBSCAN**
	NON : Suite...
Clusters de taille très variable ?	**OUI : DBSCAN · HDBSCAN · GMM**
	NON : → K-Means (solution par défaut)

A. Larhlimi

Datasets de Référence — Benchmarks

Iris

Fisher, 1936

4 features

150 échantillons

3 classes (fleurs)

Dataset classique — sépales et pétales de 3 espèces d'iris. Idéal pour tester K-Means, GMM.

Wine

UCI ML Repository

13 features

178 échantillons

3 classes (vins)

Analyse chimique de vins italiens. Tests sur spectral clustering, HAC.

Digits

NIST

64 features (8×8)

1797 échantillons

10 classes (0-9)

Images de chiffres manuscrits. Haute dimension — idéal pour UMAP+K-Means.

Implémentation Python — K-Means

```python
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

# Charger les données
X, y = load_iris(return_X_y=True)

# Créer et entraîner le modèle
kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10)
kmeans.fit(X)

# Résultats
labels = kmeans.labels_
inertia = kmeans.inertia_
print(f"Inertie : {inertia:.2f}")

# Score silhouette
from sklearn.metrics import silhouette_score
score = silhouette_score(X, labels)
print(f"Silhouette : {score:.4f}")
```

Implémentation Python — DBSCAN & GMM

```python
# DBSCAN
from sklearn.cluster import DBSCAN

db = DBSCAN(
    eps=
0.5,
    min_samples=
5
)
db.fit(X)

labels = db.labels_
n_clusters = len(set(labels))
             - (-1 in labels)

# Compter outliers
noise = list(labels).count(-1)
```

```python
# GMM
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(
    n_components=
3,
    covariance_type='full'
)
gmm.fit(X)

# Prédiction probabiliste
proba = gmm.predict_proba(X)
labels = gmm.predict(X)

# Sélection BIC
bic = gmm.bic(X)
print(f"BIC: {bic:.1f}")
```

Bonnes Pratiques en Clustering

01 Analyse exploratoire

Commencer par une visualisation PCA ou UMAP pour comprendre la structure des données avant de choisir un algorithme.

02 Normalisation des données

Standardiser (Z-score) ou normaliser (Min-Max) les features — K-Means est très sensible à l'échelle.

03 Tester plusieurs algorithmes

Comparer K-Means, DBSCAN et GMM sur les mêmes données et utiliser le score de silhouette pour évaluer.

04 Validation métier

Les métriques seules ne suffisent pas — valider les clusters avec une expertise du domaine applicatif.

05 Traitement des outliers

Identifier et traiter les outliers avant clustering — ou utiliser DBSCAN/HDBSCAN qui les gèrent nativement.

06 Reproductibilité

Fixer le random_state pour des résultats reproductibles. Utiliser plusieurs initialisations pour K-Means.

Pièges & Erreurs Fréquentes

⚠ **Ne pas normaliser les données**	K-Means et les algorithmes basés sur des distances euclidiennes sont très sensibles à l'échelle des features. Une feature avec une large variance dominera les autres.
⚠ **Choisir K arbitrairement**	Utiliser toujours la méthode Elbow, le score de silhouette ou le BIC/AIC pour choisir K — ne jamais le choisir à vue.
⚠ **Ignorer les outliers**	Les outliers faussent les centroïdes de K-Means. Détecter et traiter les outliers avant ou utiliser DBSCAN.
⚠ **Croire que plus = mieux**	Augmenter K diminue toujours l'inertie mais ne signifie pas un meilleur clustering. La méthode Elbow aide à trouver le bon équilibre.
⚠ **Utiliser K-Means sur clusters non-convexes**	K-Means assume des clusters sphériques. Pour des formes complexes, utiliser DBSCAN, Spectral ou HDBSCAN.

Conclusion

Il n'existe pas d'algorithme universellement supérieur.

Le choix dépend de la nature des données, de la connaissance a priori du problème et des contraintes computationnelles. Le clustering est autant un art qu'une science.

1 **Analyse exploratoire**

Visualisation PCA/UMAP avant tout

2 **Tester plusieurs algorithmes**

Comparer avec score de silhouette

3 **Validation métier**

L'expertise domaine prime sur les métriques

4 **Sélection finale**

Score de silhouette + cohérence métier

Algorithmes de Clustering

Classification Non Supervisée

Merci

K-Means	K-Medoids	K-Medians	DBSCAN
HDBSCAN	OPTICS	HAC	GMM
Spectral	Affinity Propagation	SOM	UMAP + K-Means

Mars 2026 · Python · Scikit-learn · 12 algorithmes présentés

A. Larhlimi